

TEAMFLOW

Systems Engineering Assignment

Architecture, Domain Model, Service Interactions
and Design Decisions

Prepared: 5 July 2026

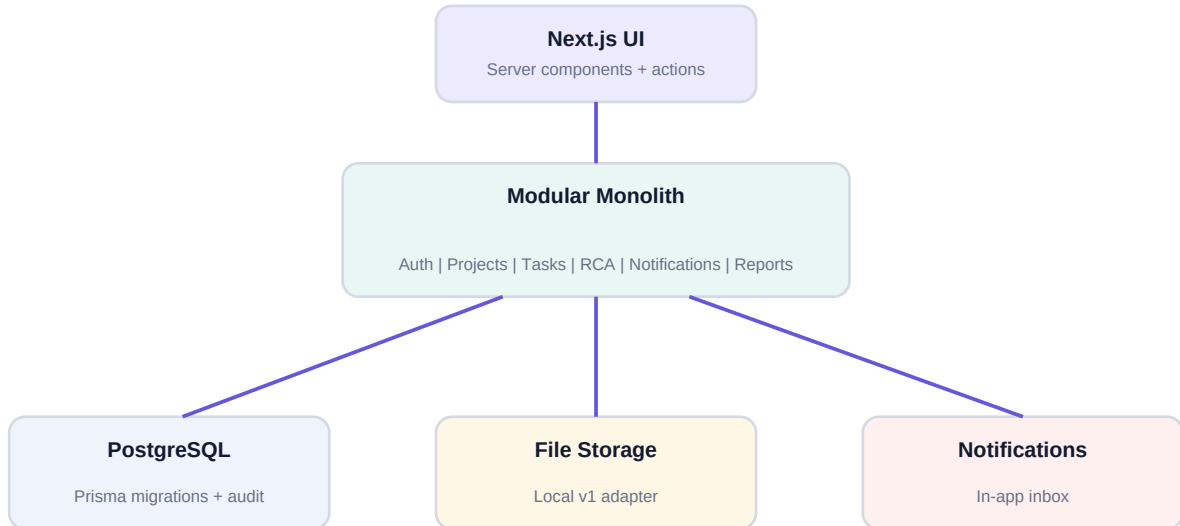
Implementation: Next.js 16, TypeScript, PostgreSQL, Prisma

Architecture: Extensible modular monolith

A unified workspace for engineering delivery, dependency-aware task execution,
structured RCA review, notifications, and live reporting.

1. Architecture

TeamFlow is one deployable Next.js application organized around explicit domain modules. Core writes are synchronous and transactional. Secondary effects are recorded as outbox events so delivery can later move to a queue without rewriting domain behavior.



Key properties

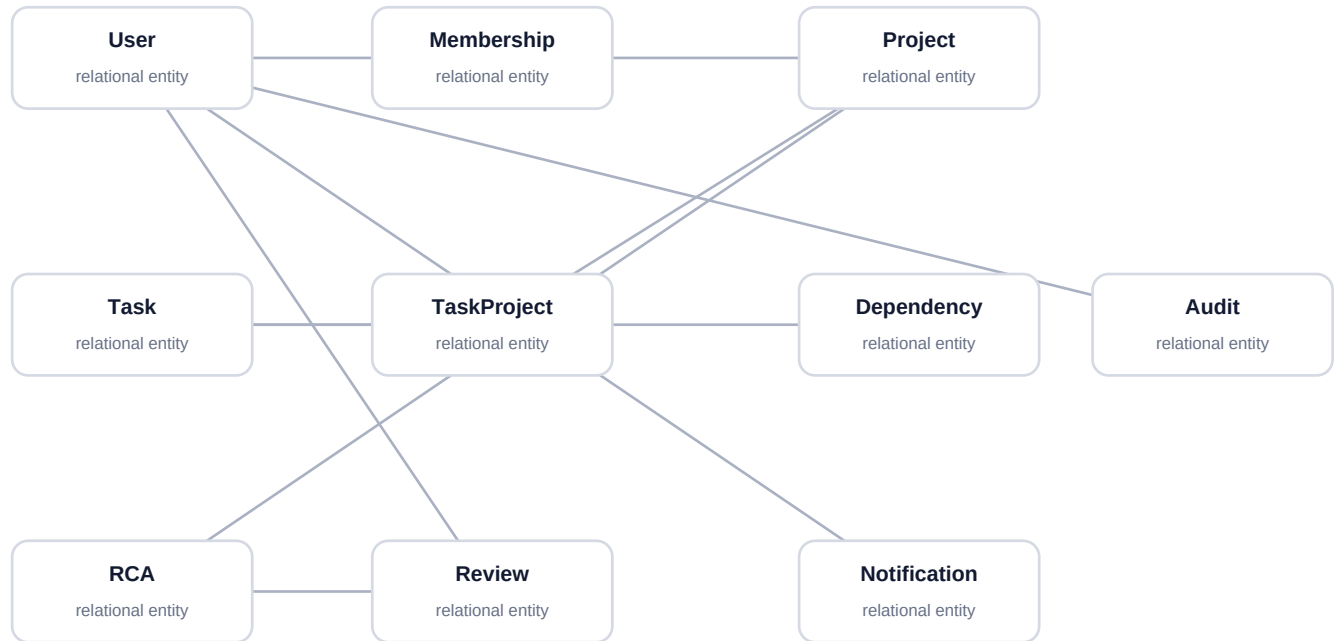
Server-side authorization protects every mutation. PostgreSQL provides relational integrity for memberships, multi-project tasks, dependencies, review assignments, notification deduplication, and audit history. Authenticated image downloads are backed by a replaceable storage interface; local disk is used for this self-contained v1.

Service interaction

Browser request -> Server action -> Session + permission check -> Domain validation -> Prisma transaction -> Aggregate + audit + outbox -> Response. Notification delivery creates deduplicated in-app records for assignments, mentions, and review requests.

2. Domain Model

Tasks retain one identity while being tracked in several projects through TaskProject. TaskDependency is a directed edge. RCA review authority comes from explicit assignments, not project role alone.



Important constraints

Composite membership and task-project keys prevent duplicates. Dependencies reject direct self-reference. Comments and attachments belong to exactly one task or RCA. Completed reviews require a decision, comment, and completion time. Notification deduplication keys are globally unique.

Workflow rules

The normal task path is To do -> In progress -> In review -> Done, with Blocked and Cancelled states plus controlled reopen and restore paths. Dependency and capacity conflicts remain warnings. Project Managers can replace or remove pending reviewers; completed review decisions remain immutable. Unanimous RCA approval permits closure, while any rejection produces Changes Required.

3. API and Interaction Overview

Decision	Alternative	Tradeoff / rationale
Sign in	External identity provider	Local script credentials create a signed HTTP-only session; provider integration remains replaceable.
Create/update task	Direct client database access	Server actions enforce membership, then write task, audit, outbox, and notification records transactionally.
RCA decision	UI-only validation	The server verifies assignment authority and requires Approved/Rejected plus a non-empty comment.
Notifications	External email dependency	Deduplicated in-app alerts keep the local v1 self-contained and require no domain or provider account.
Reporting	Precomputed warehouse	Live PostgreSQL queries keep v1 simple and current; projections can be added at scale.
CSV export	Unfiltered database dump	Export applies the active project, status, assignee, and search filters before serialization.

Implemented routes

/login credentials and session creation; **/** live dashboard; **/tasks** Kanban/list/calendar, workflow, reassignment, dependencies, comments, photos and CSV; **/rcas** structured review and reviewer reassignment; **/notifications** inbox; **/reports** live analytics; **/settings** membership and roles.

4. Design Decision Log

Decision	Alternative	Tradeoff / rationale
Modular monolith	Microservices	Lower delivery and operating cost; explicit modules preserve later extraction.
PostgreSQL + Prisma	NoSQL	Transactions, joins, constraints, migrations, and auditability fit the domain.
Synchronous core writes	Fully asynchronous	Users receive immediate validation and authoritative outcomes.
Domain events + outbox	Inline side effects	Adds a table and dispatcher boundary but decouples successful writes from providers.
Object storage adapter	Database blobs / local disk	Scalable binaries with metadata and ownership retained in PostgreSQL.
Non-blocking dependency warnings	Reject conflicting saves	Keeps planning flexible while surfacing delivery risk.
Unanimous RCA approval	Majority / first approval	Slower closure, but every assigned reviewer is accountable.
Server-enforced permissions	Client-only controls	Mutations remain safe regardless of browser behavior.

Future deployment scenarios

Offline-first: local command queue, aggregate versions, idempotency keys. **Compliance:** immutable audit export, retention and customer-managed keys. **High scale:** read replicas, partitioned audit/outbox data and reporting projections. **Multi-region:** tenant home regions and replicated reads. **Extreme low cost:** one application instance, managed PostgreSQL, and in-process outbox delivery.

5. Delivery Status and Evidence

The repository includes two PostgreSQL migrations, deterministic seed data, protected server-rendered screens, adapter interfaces, architecture documentation, and automated tests for the highest-risk rules.

Decision	Alternative	Tradeoff / rationale
Schema	Validated	Prisma client generated; migrations applied to PostgreSQL 17.
Build	Passing	Next.js production build and TypeScript checks pass.
Lint	Passing	ESLint reports no errors or warnings.
Domain tests	17 passing	Sessions, transitions, mentions, notification policy, RCA decisions, CSV filtering, and warnings.
Security	Implemented	Script password hashes, signed HTTP-only cookies, server permission checks, no committed secrets.
Known limits	Documented	External email is deferred; production object storage, identity provider, and deployment remain open.

Demonstration journey

Sign in as the seeded Project Manager; review live dashboard metrics; create and edit a task; add blockers and observe warnings; comment with @mentions; inspect notifications; submit an RCA review; view reports; manage project membership.

Repository setup and environment variables are documented in README.md. The .env file is ignored by Git; .env.example contains placeholders only.